Git and GitHub

For the following questions, consider the file system below. You begin working in a shell at the Home directory, `~`.

```
~
├── practice
│   ├── ex1.R
│   └── ex2.R
└── my-project
    ├── README.md
    └── scripts
        ├── data-cleaning.R
        └── modeling.R
```

1. If you run the `ls` command, what will the output be?

2. What commands would you run to move into `my-project` and designate that directory as a git repository?

3. When you designate an existing directory as a git repo and it already has files in it, they will be neither staged nor committed. What single command could you use to stage the existing three files? Note that you can add multiple files at once by separating them by a space (see the wedding example)[1].

4. Once all of the files are staged, write the command can be used to take a snapshot of the state of those files along with a helpful caption.

[1] There are several other ways to do this. Feel free to check online for different methods.

5. Say that the project has a Part 1 and a Part 2. These existing files accomplish Part 1. To accomplish Part 2, you add several dozen lines of new code to `data-cleaning.R` and write and save a new script called `visualization.R` right alongside it. What commands would you run to store those changes along with a helpful description?

Data Wrangling II

For this problem set, you'll be working with the `storms` tibble built into `dplyr`. This is a larger data frame than previous, so you may need to inspect it more carefully in R before proceeding with these questions. Please answer the question in common english and provide the code needed to produce the output used in your answer.

On the *challenging* exercises, a helpful approach is to start by sketching the data frame that would answer the question. What are the dimensions? What does each row refer to? Which columns are needed? Then piece together your pipeline to lead from `storms` to that data frame.

```
library(dplyr)
storms
```

```
# A tibble: 20,778 × 13
   name   year month   day  hour   lat  long status      category  wind pressure
   <chr> <dbl> <dbl> <int> <dbl> <dbl> <dbl> <fct>          <dbl> <int>    <int>
 1 Amy    1975     6    27     0  27.5 -79   tropical d…       NA    25     1013
 2 Amy    1975     6    27     6  28.5 -79   tropical d…       NA    25     1013
 3 Amy    1975     6    27    12  29.5 -79   tropical d…       NA    25     1013
 4 Amy    1975     6    27    18  30.5 -79   tropical d…       NA    25     1013
 5 Amy    1975     6    28     0  31.5 -78.8 tropical d…       NA    25     1012
 6 Amy    1975     6    28     6  32.4 -78.7 tropical d…       NA    25     1012
 7 Amy    1975     6    28    12  33.3 -78   tropical d…       NA    25     1011
 8 Amy    1975     6    28    18  34   -77   tropical d…       NA    30     1006
 9 Amy    1975     6    29     0  34.4 -75.8 tropical s…       NA    35     1004
10 Amy    1975     6    29     6  34   -74.8 tropical s…       NA    40     1002
# ℹ 20,768 more rows
# ℹ 2 more variables: tropicalstorm_force_diameter <int>,
#   hurricane_force_diameter <int>
```

6. What are the dimensions of the data frame? What does each row correspond to (i.e. what is the unit of observation)?

7. Calculate the average wind speed and pressure for each category of storm.

8. Find the number of observations and the maximum wind speed for each combination of storm status and category. Sort the results by maximum wind speed in descending order[2].

9. Use the `.by` shortcut to calculate the average pressure for each year. Repeat the calculation using `group_by()`. Do you get the same answer?

10. For each storm name, calculate the z-score of wind speed (standardized within each storm) and store it in a new column called `wind_z`. Return the data frame that includes this new column.

11. *Challenging*. For each storm category, arrange the storms in descending order by their maximum wind speed. Retain only the top three storms in each category[3].

12. *Challenging*: Create a summary that shows, for each year: the number of unique storms, the average duration (in hours) of storms, and the proportion of observations that are category 4 or 5 storms.

[2]Hint 1: `n()` is a function that returns the row count when used inside `summarize()`. Hint 2: You can group by more than one variable.

[3]Hint: If you group by multiple variables, a call to `summarize()` will peel off the outermost grouping (the last argument in `group_by()`) and keep the others

13. You may have noticed that your results for the answers in this problem set contain a value of `NA`. By default, most statistical summaries in R (`mean()`, `max()`, etc.) with return `NA` if any of the values are `NA`. If you prefer, you can calculate the mean after dropping the `NA` observations. One way to do this is to use the `na.rm = TRUE` option for functions that allow it (see `?mean`). The other way is to use the `drop_na()` function in `dplyr` (see `?drop_na()`).

    Revisit your code above and, if you haven't already, drop the `NA` values in a column before calculating summary statistics.

Quarto

The following questions address the quarto document below. Consult the Quarto Guide and the Quarto Reference for the authoritative documentation.

````
---
title: Interstellar Analysis
author: Murphy Cooper
format: typst
---

## Introduction

Readings from the radio telescope indicate unusual activity in the gamma-Q sector.

```{r}
#| echo: false
ggplot(df, aes(x = hrs, y = freq)) +
  geom_line()
```
````

14. What is the effect `echo: false`?

15. Which line would you change if you wanted this document to render to an HTML file?

16. If you wanted to display the code but not actually run it, write option setting(s) that you would include at the top of the code cell.

17. A classmate copies this code cell into their R console, where it runs without complaint and makes the plot. But when they run `quarto render` on the document, they get the error `could not find function "ggplot"`. Why does the code run in the console but fail during rendering? What would you add to the document to fix it?

Practice with `hello-penguins.qmd`

These exercises are hands-on. Download `hello-penguins.qmd` from Ed (it's the same file we used in class), open its folder in Positron, and run `quarto preview hello-penguins.qmd` so the document re-renders every time you save. Look things up in the Quarto Guide and the Quarto Reference as you go. Many of these are features you'll want in your project reports (the website's search tool is also helpful).

Essentials

18. Add an `author` and a `date` to the metadata. Then change the HTML `theme` and add a table of contents.

19. The text in the Species section says the figure is a bar plot. Read the code and fix the description so it matches what the figure actually shows. While you're at it, give the section a more descriptive *header*.

20. The first code cell prints package startup *messages* into your report. Use *cell options* to hide both the messages and the code in that cell, while still running it.

21. Give the scatterplot a *caption* and change its width and height. Then give the cell a label so you can *cross-reference* the figure, and change the prose so it says something like "Figure 1 shows…" with the figure number generated automatically (e.g. `@fig-bills`).

22. The `gt` package is loaded but never used. Pipe the penguins table into `gt()` so it renders as a formatted table. Give it a caption and a label, and cross-reference it in the text the same way you did the figure.

23. Add a sentence to the Data section that reports how many penguins are in the dataset. Instead of typing the number, use *inline code* so the number is computed from the data when you render.

Showing off

24. Add `code-fold: true` to the HTML format options. What changes in the rendered document?

25. Wrap the figure and the table in a *tabset* so readers can click between two tabs, one labeled “Plot” and one labeled “Table”.

26. Give the document a light theme and a dark theme so readers get a toggle switch in the upper right corner.

27. Install the `plotly` package, then wrap your scatterplot in `plotly::ggplotly()`. Hover over the points, then click and drag. What do you get in the HTML output that a static figure can’t do?

28. Change the format to `revealjs`, then use `##` headers so the figure and the table each get their own slide. Change the transition style.

Grammar of Graphics

Each of the following questions should be answered using the `storms` data set built into `dplyr`. Provide the code to construct each of the following plots and make a sketch of the resulting plot. No need to enumerate all of the detail of the data in the plot but focus on the general shape and be sure to be clear what all axes and legends are (i.e. what the aesthetic mappings are).

29. Plot the distribution of the wind speed three ways: as a histogram, a box plot, and a density curve.

30. A bar chart showing the distribution of storm status found across all records in the data set.

31. A bar chart showing the distribution of storm status of when considering just one record for each named storm[4].

[4]There are several ways to reduce the data frame to contain only one row for each named storm. The simplest is probably to group the data frame, then slice it and take the first record

32. A scatterplot of the relationship between wind speed and pressure for every record in the data set.

33. A scatterplot of the relationship between average wind speed and average pressure for each storm.

34. A line plot of the relationship between wind speed and pressure for all of the storms in 2014. Each storm should be indicated with a different color[5].

35. Create a line plot that shows the average duration of storms over time. That is: the x-axis should be the year and the y-axis should be average storm duration.

[5]If you'd like to a slightly more sophisticated version of this plot, read about `geom_path()`. Instead of lines, this can plot paths with an arrowhead indicating the progression of that storm.

Data Cleaning

The following code creates a small, messy data frame of survey responses from students.

```
survey <- tibble::tibble(
  `Student Name` = c("Ana", "Ben", "Cam", "Dev", "Eli", "Fay",
                     "Gus", "Hana", "Ivy", "Ivy", "Jon", "Kai"),
  AGE = c(19, 20, -99, 21, 250, 19, 67, 20, 22, 22, 18, 20),
  Campus = c("Berkeley", "berkeley", "Berkeley ", "UCB", "Berkeley", "Laney",
             "Berkeley", "UC Berkeley", "Berkeley", "Berkeley", "UCB", "berkeley"),
  commute_min = c("15", "N/A", "30", "", "10", "45",
                  "20", "N/A", "5", "5", "25", "12"),
  steps = c("8,200", "1,200", "12,050", "6,000", "9,300", "4,750",
            "10,400", "7,800", "3,100", "3,100", "11,000", "5,500")
)
```

Audit `survey` by working through the data cleaning checklist. For each question, give the code you used and answer in full sentences.

36. **Step 1: Source.** What appears to be the unit of observation in `survey`? What are the variables? There is no data dictionary for `survey`, so do your best given what you can see here.

37. **Step 2: Names.** Which column names are non-syntactic, and which ones break the snake case convention? Write a `rename()` call that gives every column a syntactic, snake case name.

38. **Step 3: Types.** Which columns have a type that doesn't match what they mean? Why?

39. **Step 4: Missing values.** What does `colSums(is.na(survey))` return? Explain why that output is misleading. List every disguised missing value in the data and the column it's in, then write code that turns each one into `NA`.

40. **Step 5: Distinct values.** How many distinct values does `Campus` have? How many are left after you apply `trimws()` and `tolower()`? Use `case_when()` to recode the column so that each campus has exactly one label.

41. **Step 6: Ranges and outliers.** Two of the ages stand out. For each one, say whether it appears to be a *data error* or an *outlier*, and explain why. Write a `stopifnot()` call that encodes a validity constraint on age. Then say what you'd do with each value.

42. **Step 7: Duplicates and keys.** Does any row appear more than once? Show the code that checks, and write code that removes exact duplicates. Is the student's name a key for `survey` before you remove the duplicates? What about after? Show the code you used to check.

43. **Step 8: Structure.** Is `survey` tidy? Justify your answer using the definition of tidy data.

44. **Record.** Combine your fixes from the previous questions into one pipeline that starts with `survey` and returns a cleaned data frame.[6]

45. **Decide.** Pick two lines of your pipeline that a reasonable analyst might write differently. For each one, describe the alternative and explain how that choice could change the conclusions of an analysis. (For example: does `""` in `commute_min` mean "didn't answer" or "no commute"?)

[6] To convert text like `"8,200"` into a number, use `parse_number()` from the `readr` package. You can call it without loading the package by writing `readr::parse_number()`.

Reading and Writing Data

For each of the following, answer in full sentences and provide the code that you used.

46. For each file below, say whether it is plain text or binary, name a function that reads it into R, and give the type of R object you'd get back.

File	Plain text or binary?	Function	R object
`playlist.txt`			
`budget.xlsx`			
`config.yml`			
`orders.csv`			
`history.json`			

The following questions refer to a file called `songs.csv`. Download `songs.csv` to your computer

47. Open `songs.csv` in a simple text editor or print it with `cat songs.csv` in the shell (don't open it with Positron or read it into R). Why is each value in the `streams` column wrapped in quotes, while the values in `year` are not? What would go wrong when reading the file if the quotes were removed?

48. Before running any code, predict the type of each of the four columns when `songs.csv` is read with `read.csv()` and when it is read with `readr::read_csv()`. Then check your predictions using `str()`. Which columns differ between the two functions, and why?

49. Run the following code.

```
songs <- read.csv("songs.csv")
write.csv(songs, "songs-copy.csv")
read.csv("songs-copy.csv")
```

How does the data frame you read back differ from `songs`? Explain where the difference came from, then show two different ways of writing `songs` to a CSV so that it reads back with the same columns it started with.

50. The code below writes a YAML file describing one day of the course schedule.

```
writeLines(c(
  'topic: "Reading and Writing Data"',
  "docs:",
  "  - type: Slides",
  "    href: 16-reading-writing/slides.qmd",
  "  - type: Problem Set",
  "    href: problem-sets/ps6.qmd"
), "day.yml")
```

Read `day.yml` into R as `day` and use `str()` to describe its structure. Then write one line of code that extracts the string `"Problem Set"` from `day`. Finally, add a new element called `note` to `day` with the value `"Request your data exports"`, write it back to `day.yml`, and show the new contents of the file.

51. Read `songs.csv` with `readr::read_csv()`, then convert the result to JSON with `jsonlite::toJSON(songs, pretty = TRUE)`. Sketch the JSON for the first song. Look at the third object: what happened to the missing `year`? What do you get back for that value when you read the JSON back into R with `fromJSON()`?

52. Read the following description of a student.

> Jason is a junior at UC Berkeley. He is majoring in Economics and Data Science. He is taking three classes this semester: STAT 133, DATA 104, and MATH 104. The first two classes are M/W/F classes; the third is Tu/Th.

By hand (no computer), write out the contents of a YAML file, `jason.yml`, that stores this information. Then write out a JSON file, `jason.json`, that stores the same information with the same structure. Use keys that describe each piece of information, and choose a structure that keeps each class together with the days it meets[7].

[7] See the slides for a side-by-side comparison of YAML and JSON syntax.